FAIR LOAD-SHEDDING WITH QUANTUM OPTIMISATION

HACKATHON REPORT

QISKIT FALL FEST 2026

ENERGY & SMART INFRASTRUCTURE · CVaR-QAOA IN QISKIT

Team: [team name / members]

Code: github.com/usmansaeedkhosa2006/qiskit-load-shedding-qaoa

Video: [Google Drive link]

4,096 POSSIBLE SCHEDULES.
ONLY 2 FOLLOW EVERY RULE.
QAOA FOUND THE BEST ONE,
~197× MORE OFTEN THAN GUESSING.

Best of 2048 shots at depth p = 8 = exact optimum · 108 optimal shots · 12 qubits, Qiskit 2.6

CONTENTS

The Challenge: Keeping the Lights On, Fairly 4

Who Does This Help? 5

From Grid to Qubits: the Formulation 6

The Four Rules 7

Why QAOA? 8

How It Fits Together: Architecture 9

Building It in Qiskit 10

Results: Did It Work? 11

More Layers, Better Odds 12

What the Quantum Computer Saw 13

The Limits: an Honest Assessment 14

What's Next, Reproducing It and Credits 15

THE CHALLENGE:

KEEPING THE LIGHTS ON, FAIRLY

When generation can't meet demand, a power distribution company (DISCO) such as LESCO or K-Electric has to shed load: it switches whole feeders off for an hour at a time. Every evening a control room decides which neighbourhoods go dark, and in what order.

WHEN SUPPLY FALLS SHORT

Shed too little and grid frequency drops, protection relays trip, and a local shortfall can become a wide blackout. Shed too much and paying customers lose power for nothing. Cut a hospital or a water-pumping station and the harm is no longer just inconvenience.

Pakistan's DISCOs already use loss-based load management: feeders with higher losses (theft and non-payment) get longer outages. That turns a technical problem into a social one.

WHY IT'S HARD

Rotation tables are still largely built by hand from fixed categories. With hundreds of feeders, a deficit that changes every hour, protected loads, and a fairness expectation that nobody should be cut twice in a row, the number of possible schedules explodes: 2 to the power (feeders × hours).

OUR IDEA

Treat each “feeder f is off in hour h” decision as a qubit. Write every goal as an energy term, so the lowest-energy state of a quantum system is the fairest valid schedule. Then use QAOA, a hybrid quantum-classical algorithm, to search for it in Qiskit, and check the answer against an exact solver.

OUR TEST GRID

Illustrative evening peak, 18:00–20:00

FEEDER	TYPE	MW	LOSS
F1	Residential, high-loss	30	35%
F2	Mixed use	20	25%
F3	Commercial market	20	10%
F4	Residential, low-loss	10	5%
F5	Hospital + water pumping	25	never cut

Deficit to shed: 18:00 → 30 MW 19:00 → 30 MW 20:00 → 40 MW

12 qubits · 4,096 schedules · only 2 valid

WHO DOES THIS HELP?

A fair, automatic rotation table creates value for at least **four groups:**



GRID OPERATORS

A valid rotation table in seconds, not hours

Never under-shed: no frequency collapse

Re-plan instantly when the deficit changes



PAYING CUSTOMERS

Low-loss feeders cut least

No needless over-shedding

Never two outages back to back



CRITICAL SERVICES

Hospitals and water pumping are never cut

Enforced before optimisation, so it cannot fail



THE REGULATOR

Rules written as explicit maths

Auditable, repeatable, and the same for everyone

BUILT FOR ONE REAL SITUATION: THE EVENING PEAK IN A PAKISTANI DISTRIBUTION COMPANY

FROM GRID TO QUBITS

HOW WE TURNED LOAD-SHEDDING INTO A QUBO

DECISION VARIABLES

x[f,h] ∈ {0,1} equals 1 when sheddable feeder f is switched off in hour h. With 4 sheddable feeders × 3 hours that is 12 binary variables, and each one becomes one qubit.

PROTECTED FEEDERS: ZERO QUBITS

The hospital and water-pumping feeder F5 is removed before the quantum step, so x is fixed to 0. A hard rule enforced classically costs no qubits and can never be violated.

UNITS

Loads and deficits are expressed in 10 MW blocks, which keeps the penalty coefficients small and the energy landscape smoother.

THE OBJECTIVE

$$\text{minimise}\quad C(x) = \sum_f c_f \cdot x[f,h] + A \cdot \sum_h \left(\sum_f L_f \cdot x[f,h] - D_h\right)^2 + B \cdot \sum_f \sum_h x[f,h] \cdot x[f,h+1]$$

- **Harm:** $c_f = L_f \cdot (1 - \text{loss}_f)$, the MW of paying customers cut. High-loss feeders are cheaper to shed, which reproduces loss-based load management.
- **Deficit:** penalises both under-shedding (grid risk) and over-shedding (needless outages). A = 3.
- **Fairness:** a pairwise penalty on the same feeder in consecutive hours, a natural ZZ interaction. B = 3.

CHECKING THE PENALTIES

We verified A and B by brute force over all 4,096 states. The global minimum is a schedule that satisfies every rule, and the cheapest rule-breaking schedule (9.35) costs more than either valid one (7.70 and 8.00).

FROM QUBO TO ISING

Substituting $x = (1 - Z)/2$ gives a Hamiltonian $H = \sum h_i Z_i + \sum J_{ij} Z_i Z_j$, built as a Qiskit SparsePauliOp. A unit test checks that its diagonal equals C(x) for every state.

BY THE NUMBERS

12 qubits

4,096 possible schedules

2 valid schedules

12 + 26 Z and ZZ terms

THE FOUR RULES

WHAT EVERY LOAD-SHEDDING SCHEDULE MUST GET RIGHT

These are the goals a control room juggles every evening. Each one becomes a term in the quantum cost function, or is removed before the quantum step. Together they create the trade-off the optimiser has to resolve.

1



MEET THE DEFICIT

Shed exactly the missing megawatts each hour, no more and no less. Modelled as a squared penalty $A \cdot (\text{shed} - \text{deficit})^2$, so both under- and over-shedding cost energy.

2



LOSS-BASED PRIORITY

Prefer feeders with high technical and commercial losses. Each feeder-hour costs $L \cdot (1 - \text{loss rate})$, the paying-customer MW cut, as a linear term.

3



PROTECT CRITICAL LOADS

Hospitals and water pumping are never shed. This is a hard rule handled classically: the feeder is dropped from the model and uses no qubits.

4



NO BACK-TO-BACK CUTS

Fairness: no feeder goes dark in two consecutive hours. This is a pairwise penalty $B \cdot x[f,h] \cdot x[f,h+1]$, which maps to one ZZ interaction per pair.

WHY QAOA?

THE DESIGN CHOICES BEHIND OUR QUANTUM SOLVER

1



A NATURAL FIT

Our cost has only linear and pairwise terms: exactly what QAOA's cost layer (RZ and RZZ gates) implements, with no extra encoding.

2



QUANTUM DOES THE WORK

Every measurement is a candidate rotation table. The circuit's output is the solution itself, not a side calculation.

3



NEAR-TERM FRIENDLY

Shallow parameterised circuits plus a classical optimiser suit today's noisy hardware, and run on a laptop simulator at 12 to 15 qubits.

4



CVAR OBJECTIVE

We minimise the mean cost of the best 10% of outcomes, not the average, because in optimisation only the best samples matter (Barkoutsos et al., 2020).

5



INTERP WARM STARTS

Depth p+1 is initialised by interpolating the trained angles from depth p (Zhou et al., 2020), with 10 random restarts per depth.

6

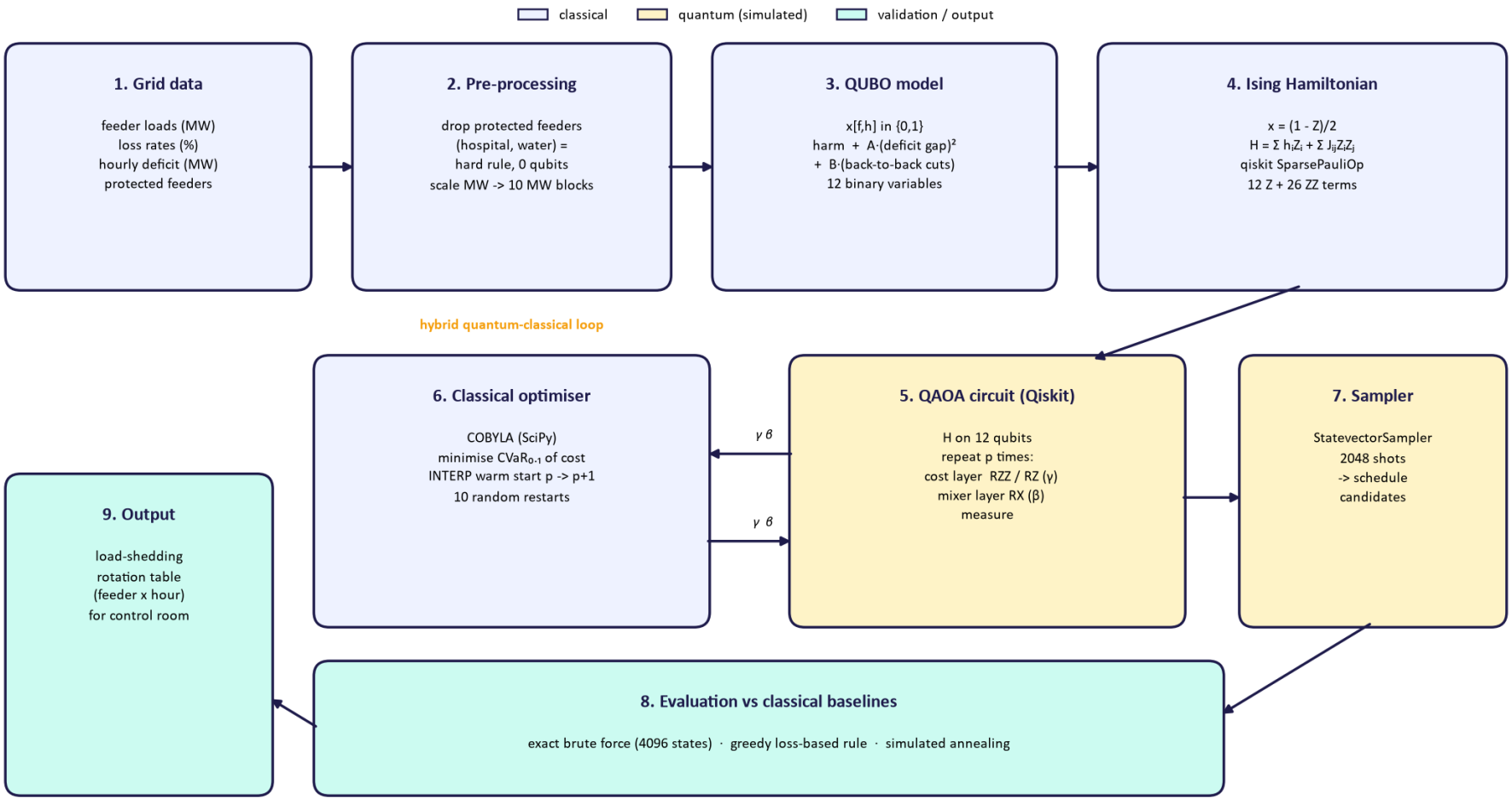


ALTERNATIVES CONSIDERED

VQE (no problem structure, more parameters), Grover adaptive search (deep circuits and an oracle), quantum annealing (not in Qiskit).

ARCHITECTURE

HOW IT FITS TOGETHER



Classical pre-processing → QUBO / Ising → hybrid QAOA loop (quantum circuit ⇄ classical optimiser) → sampling → validation against exact and heuristic baselines → rotation table.

IMPLEMENTATION

BUILDING IT IN QISKIT

CODE LAYOUT

- **instance.py:** feeder data and deficits
- **qubo.py:** QUBO builder, Ising mapping, schedule metrics
- **qaoa.py:** QAOA circuit, CVaR, COBYLA loop, Qiskit Sampler
- **baselines.py:** brute force, greedy rule, simulated annealing
- **run_experiments.py:** the full study, end to end
- **tests/:** Ising = QUBO, optimum is valid, simulator equivalence

STACK

Qiskit 2.6.2 (QuantumCircuit, ParameterVector, SparsePauliOp, Statevector, StatevectorSampler, transpile, GenericBackendV2), SciPy COBYLA, NumPy and Matplotlib. Python 3.14.

THE QUANTUM LOOP

For each depth p = 1…8: build the circuit (H on all qubits, then p rounds of an RZZ/RZ cost layer and an RX mixer). COBYLA tunes the 2p angles for up to 400 iterations from 10 starting points. The trained circuit is sampled 2,048 times with Qiskit's Sampler primitive.

CIRCUIT COST ON A DEVICE

Compiled to a 3×4 grid of qubits with native {CZ, RZ, SX, X} gates:

DEPTH	RZZ	CZ	DEPTH AFTER TRANSPILE
p=1	26	75	115
p=8	208	731	762

HONESTY NOTE

During training, the QAOA state is computed with an equivalent NumPy routine: the cost layer is a diagonal phase and the mixer is RX on every qubit. Rebuilding the Qiskit circuit on every optimiser call was too slow.

A test confirms the routine matches qiskit.quantum_info.Statevector to 1e-16.

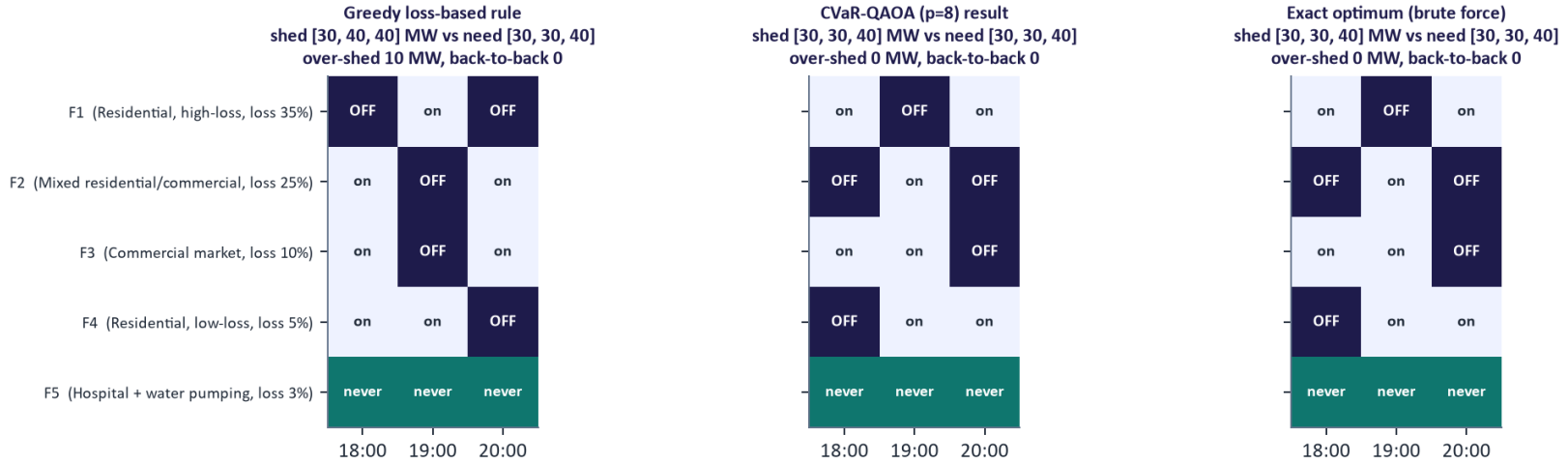
Every probability, sample and figure in this report comes from the actual Qiskit circuit.

REPRODUCIBLE

All seeds are fixed. The full run takes about 6 minutes on a laptop.

RESULTS:

DID IT WORK?

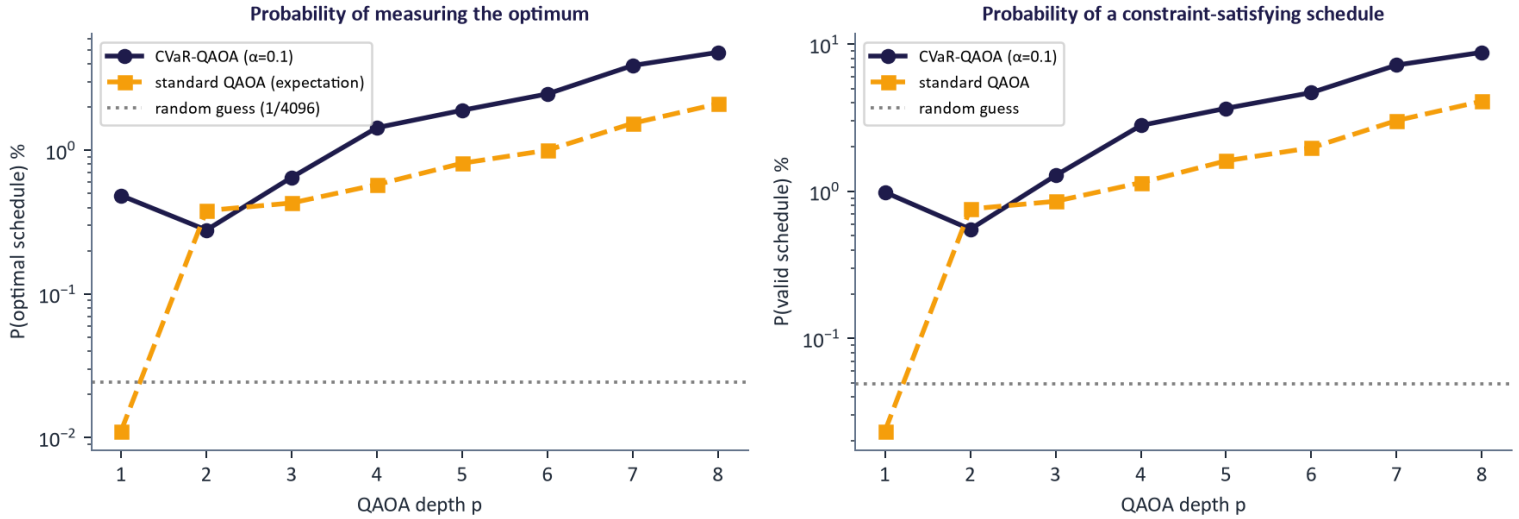


METHOD	FEEDERS OFF 18:00 | 19:00 | 20:00	COST	OVER-SHED MW	UNMET MW	BACK-TO-BACK	VALID
Exact (brute force)	F2+F4 | F1 | F2+F3	7.70	0	0	0	yes
CVaR-QAOA (p=8)	F2+F4 | F1 | F2+F3	7.70	0	0	0	yes
Greedy loss-based rule	F1 | F2+F3 | F1+F4	11.15	10	0	0	no
Simulated annealing	F2+F4 | F1 | F2+F3	7.70	0	0	0	yes

QAOA's best sample is the exact optimum: it sheds the high-loss feeder F1 at the 19:00 peak and rotates the others around it. The greedy rule over-sheds 10 MW at 19:00 and cuts 81.5 paying-customer MW against 77.0 for the optimum.

QAOA PERFORMANCE

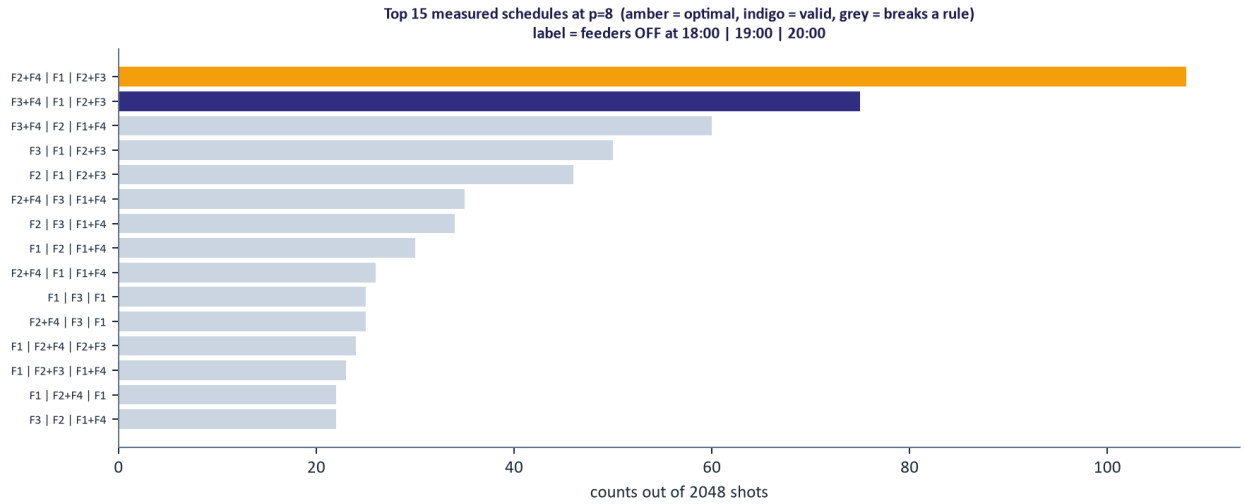
MORE LAYERS, BETTER ODDS



P	P(OPTIMUM)	VS RANDOM	P(VALID)	APPROX. RATIO	OPTIMAL SHOTS	MOST FREQUENT = OPTIMUM	STANDARD QAOA P(OPT)
1	0.48%	20×	0.99%	0.760	9 / 2048	no	0.01%
2	0.28%	11×	0.55%	0.921	6 / 2048	no	0.38%
3	0.65%	27×	1.28%	0.947	11 / 2048	no	0.43%
4	1.44%	59×	2.80%	0.929	35 / 2048	no	0.58%
5	1.90%	78×	3.67%	0.956	35 / 2048	no	0.81%
6	2.47%	101×	4.69%	0.955	58 / 2048	yes	1.00%
7	3.90%	160×	7.22%	0.933	94 / 2048	yes	1.54%
8	4.82%	197×	8.81%	0.955	108 / 2048	yes	2.11%

Random guessing finds the optimum 0.024% of the time. From p = 6, the single most frequently measured schedule is the optimum.

WHAT THE QUANTUM COMPUTER SAW



Most frequent schedules at p = 8 out of 2,048 shots. Amber = optimal, indigo = valid, grey = breaks a rule.

Most measured schedules still break a rule, usually by being 10 MW short in one hour. That is the price of soft penalties. But the optimum is clearly the tallest bar, so reading off the most frequent result already gives the right answer.

SCALING TO 15 QUBITS

Adding a fifth feeder gives 15 qubits, 32,768 schedules and 15 valid ones.

P	P(OPT)	P(VALID)	BEST = OPT
1	0.026%	0.26%	no
2	0.067%	0.60%	no
3	0.065%	0.60%	yes
4	0.057%	0.75%	yes

The optimum appeared only once in 2,048 shots. Penalty-based QAOA degrades fast as the problem grows. This is our main limitation.

THE LIMITS

AN HONEST ASSESSMENT OF WHAT DIDN'T WORK

1

NO QUANTUM ADVANTAGE

Brute force solves 12 qubits in milliseconds and simulated annealing finds the optimum instantly. Real DISCOs would use MILP solvers. We claim no speedup.

2

LOW SUCCESS RATE

Even at the best depth only 4.82% of shots are optimal. We rely on best-of-many samples, which scales poorly.

3

PENALTY TUNING

A and B were hand-tuned. The cost range (7.7 to 240) makes the landscape rugged. Constraint-preserving XY mixers would be better.

4

EXACT-MATCH DEFICITS

The squared penalty needs sheds to match deficits exactly in 10 MW blocks. Real deficits need slack-variable inequalities, which cost more qubits.

5

NOISELESS SIMULATION

Statevector simulation only. After routing, the p = 8 circuit needs about 730 CZ gates, and today's hardware noise would swamp it.

6

TRAINING COST

Thousands of circuit evaluations per depth: the classical optimiser loop is the bottleneck, not the quantum circuit.

7

ILLUSTRATIVE DATA

Feeder data are hypothetical. The model ignores network topology, voltage limits and customer counts per feeder.

8

WHAT DIDN'T WORK

At 15 qubits QAOA almost never found the optimum. Standard QAOA beat CVaR only at p = 2, and at p = 1 it essentially never found a valid schedule.

WHAT'S NEXT

NEXT STEPS

- **XY mixers:** keep the fairness rule satisfied by construction.
- **Decomposition:** split by hour or grid station so each quantum sub-problem stays small.
- **Warm starts:** seed QAOA from the greedy schedule.
- **Hardware:** run on IBM devices with error mitigation.
- **Richer model:** customer counts, priority classes and multi-day fairness.

REPRODUCE IT

```
git clone

https://github.com/usmansaeedkhosa2006/qiskit-load-shedding-qaoa.git
cd qiskit-load-shedding-qaoa
python -m venv .venv
pip install -r requirements.txt
python -m tests.test_formulation
python run_experiments.py
```

Results, figures and this report are regenerated by run_experiments.py, make_figures.py and make_report.py.

CREDITS

All code was written by the team for this hackathon. Ideas we used:

- QAOA: Farhi, Goldstone & Gutmann, arXiv:1411.4028 (2014)
- CVaR objective: Barkoutsos et al., Quantum 4, 256 (2020)
- INTERP initialisation: Zhou et al., Phys. Rev. X 10, 021067 (2020)
- QUBO penalties: Lucas, Front. Phys. 2, 5 (2014)
- Software: Qiskit, SciPy, NumPy, Matplotlib

THANK YOU

Code: github.com/usmansaeedkhosa2006/qiskit-load-shedding-qaoa

Video: [Google Drive link]

Team: [team name / members]

